

Version 1.2

August 2026



MACHINE LEARNING

MODULE 2 - TEXT CLASSIFICATION

Authored by:

Ir. Yufis Azhar, S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

UNIVERSITY OF MUHAMMADIYAH MALANG

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE OBJECTIVES.....	3
PREPARATION.....	3
MAIN TOPICS.....	4
A. DEFINITION OF TEXT DATA.....	4
B. APPLICATIONS OF TEXT DATA PROCESSING.....	5
C. PREPROCESSING STAGES FOR TEXT DATA.....	5
SIMPLE SIMULATION.....	9
CODELAB (10%).....	12
LAB ASSIGNMENT (90%).....	15



INTRODUCTION

OBJECTIVES

This module aims to help students understand the principles and methods of text classification. Students will learn how to process text data, design classification models, and apply sample code during the module activities.

MODULE OBJECTIVES

Students will understand the concepts covered in the module, create deep learning models using the Keras library or other libraries to classify text datasets, and complete the practical module activities.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Google Collaboratory.
3. Library NumPy, Pandas, Matplotlib, Sklearn, and Tensorflow.
4. Source Code : [🔗 MODULE 2 MACHINE LEARNING - TEXT CLASSIFICATION](#)



MAIN TOPICS

In today's digital age, the volume of text-based data is steadily increasing as communication technology and social media develop. Text-based information is available in various forms, such as emails, social media comments, news articles, product reviews, and more. To use this text data effectively, techniques are needed to group or classify it automatically. One commonly used solution is text classification.

A. DEFINITION OF TEXT DATA

Text data is a collection of information presented as text, such as sequences of words, sentences, or paragraphs that convey a specific meaning. This information can come from various sources, such as documents, articles, social media posts, and so on. Here are some examples:



Figure 1. Examples of Social Media Comment
Source: Agora Pulse

Text data can take various forms and formats, depending on the context and application. Examples include :

1. Text-based documents, such as reports, books, or papers.
2. Digital communications, such as emails, instant messages, and tweets.
3. Transcripts of conversations or interviews.



B. APPLICATIONS OF TEXT DATA PROCESSING

Text data is used in various sectors, such as finance, insurance, healthcare, and others. Here are some examples of text data processing applications :

1. Spam Filters

One of the most annoying things about email is spam. These spam filters scan the text in the emails we receive and try to figure out what they mean to determine whether they're spam.

2. Sentiment Analysis

Sentiment analysis can gauge customer opinions, satisfaction levels, and market response to products by analyzing social media posts, customer reviews, and survey responses. This helps companies make decisions, especially when formulating marketing strategies.

3. Chatbot and Virtual Customer Support

Text data processing enables chatbots to understand and respond to customer questions and comments in a conversational way. This technology is widely used in customer service.

4. Automatic Translation

Machine translation is the process of automatically converting text from one language to another using a computer. This technique is very useful for overcoming language barriers in global communication.

5. Automatic Summary

Automatic summarization is the process of automatically generating a summary from a long text. The goal is to present the main information from the original document more concisely, without omitting key points.

C. PREPROCESSING STAGES FOR TEXT DATA

Text data preprocessing is a key step in preparing data for machine learning or natural language processing (NLP) models. Here are 10 common steps in the text data preprocessing process.

1. Removing Punctuation and Special Characters

Punctuation marks such as periods, commas, exclamation points, and special characters such as @, #, or numbers are often unnecessary for text processing. This is because these characters can introduce noise and interfere with the model's learning process.

```
hello world @2024 amazing ! ==> hello world amazing
```

Figure 2. Examples of Removing Punctuation and Special Characters



2. Removing Extra Spaces

These extra spaces usually appear due to typos or when copying text from another source.

```
" Saya suka Data Mining " ==> "Saya suka Data Mining"
```

Figure 3. Examples of Removing Punctuation and Special Characters

3. Converting Text to Lowercase

Computers distinguish between uppercase and lowercase letters, so without converting to lowercase, "Data" ≠ "data" and "AI" ≠ "ai."

4. Tokenization

Breaking text down into smaller units, usually words or sentences. Each word becomes a token that can be processed further.

```
Hello world amazing Python ==> Hello, world, amazing, Python
```

Figure 4. Examples of Tokenization

5. Stopwords Removal

Stopwords are common words that frequently appear in language but have no significant meaning in the context of analysis, such as "which," "and," and "at." These words are often removed to reduce noise.

```
Saya belajar di rumah dengan teman saya ==> belajar rumah teman
```

Figure 5. Examples of Stopwords Removal

6. Stemming and Lemmatization

a. Stemming

The process of converting a word to its base form by removing prefixes or suffixes according to specific rules, regardless of the word's linguistic structure.

```
berlari, lari-lari, pelari ==> lari  
makanlah, memakan, dimakan ==> makan
```

Figure 6. Examples of Stemming



b. Lemmatization

The process of converting a word into its base form (lemma) while taking into account the word's grammatical structure and meaning.

```
anak-anak ==> anak
bermain ==> main
```

Figure 7. Examples of Lemmatization

7. Feature Engineering

In Natural Language Processing (NLP), vectors play a crucial role in transformation. Processed text is converted into a numerical representation so machine learning models can use it. This process typically uses a vectorizer.

A vector is essentially a numerical representation of a word, phrase, or even an entire document. These vectors are not merely numbers; they also capture the text's meaning (semantics) and grammatical properties (syntax), allowing machines to perform mathematical operations on them. With this representation, algorithms can perform tasks such as classification, clustering, and prediction. Some common techniques used include:

- a. **One-Hot Encoding:** Represents words as binary vectors.
- b. **Bag of Words (BoW):** Represents text by counting the number of times a word appears in a document.
- c. **Text Vectorization:** Converts raw text into a numerical representation (vector).
- d. **Term Frequency-Inverse Document Frequency (TF-IDF):** Measures the importance of a word in a document relative to the corpus.
- e. **CountVectorizer:** A practical implementation of Bag of Words in scikit-learn.
- f. **Word Embeddings:** Using numerical vectors to represent words, such as Word2Vec or GloVe.

8. Data Splitting

Split the dataset into training, validation, and test sets, if necessary.

9. Normalization or Standardization

In text data, normalization converts non-standard words, abbreviations, or spellings into standard or commonly used forms.



```
gk ngerti          ==> tidak mengerti  
kamu udh mkn?      ==> kamu sudah makan?  
aq mau pgn bgt jln2 ==> aku mau pingin banget jalan-jalan  
dmn tmn2 skrg?     ==> di mana teman-teman sekarang?
```

Figure 8. Examples of Normalization

10. Storage or Serialization

The processed dataset can be saved in an appropriate format, such as CSV or JSON, or directly converted into an input vector for use in a model.



SIMPLE SIMULATION

To classify text, we can use a simple model with Sklearn. In this simulation, we will create and train a model using a simple neural network, such as Naive Bayes. Here are the steps:

1. Setting Up the Environment and Importing the Required Libraries

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
```

An RNN (Recurrent Neural Network) is a type of artificial neural network specifically designed to process sequential data, such as text, speech, music, or time series.

Unlike standard feedforward neural networks, which process inputs only once, RNNs have “memory”:

- The output from the previous step can affect the calculations in the next step.
- In other words, an RNN can remember contextual information from the past.

2. Dataset Preparation

Here is an example of a simple dataset.

```
texts = [
    "I love 123 this movie !!!",
    "This is https://fufufafa.com an amazing film ???",
    "I hated the movie @sahroni",
    "This was the 000 worst film ever !!??",
    "What a great movie :-)",
    "This film is terrible 1312 !!!",
    "Absolutely fantastic @windah",
    "I really 123 enjoyed this film https://soundhoreg.com",
    "I did not like this movie #badmovie",
    "It was a total waste of time !!! 000",
    "One of the best movies ever :-)",
    "This movie is boring ???",
    "Would recommend it to everyone https://apalah.com",
    "I will never watch this film again @greenpink"
]

labels = [
    "positive", "positive", "negative", "negative", "positive", "negative", "positive", "positive",
    "negative", "negative", "positive", "negative", "positive", "negative"
]
```



3. Data Preprocessing

```
# Encode Label
le = LabelEncoder()
y = le.fit_transform(labels)

# Split data
X_train_txt, X_test_txt, y_train, y_test = train_test_split(texts, y, test_size=0.30,
random_state=42, stratify=y)

# Step 1-6 bagian TAHAPAN PREPROCESSING PADA DATA TEXT
def custom_standardize(s):
    s = tf.strings.lower(s)
    s = tf.strings.regex_replace(s, r"http\S+|www\.\S+", " ")
    s = tf.strings.regex_replace(s, r"@|w+|#\w+", " ")
    s = tf.strings.regex_replace(s, r"^[a-z\s]", " ")
    s = tf.strings.regex_replace(s, r"\s+", " ")
    return tf.strings.strip(s)

# Vectorizer (ubah teks ke representasi angka)
VOCAB_SIZE = 2000
MAX_LEN = 12

vectorizer = layers.TextVectorization(
    standardize=custom_standardize,
    max_tokens=VOCAB_SIZE,
    output_mode="int",
    output_sequence_length=MAX_LEN
)
vectorizer.adapt(np.array(X_train_txt))

# Ubah teks -> urutan indeks kata (siap ke Embedding)
X_train = vectorizer(np.array(X_train_txt)).numpy()
X_test = vectorizer(np.array(X_test_txt)).numpy()
```

TextVectorization is a Keras layer for preprocessing text (cleaning, tokenization, integer encoding, padding) so a neural network model can use raw text directly.

4. Building & Training Models

```
# Model SimpleRNN
model = models.Sequential([
    layers.Embedding(VOCAB_SIZE, 32, input_length=MAX_LEN),
    layers.SimpleRNN(32),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.summary()
```



```
# Training Model
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=25, batch_size=4, verbose=0
)
```

5. Model Evaluation

```
# Evaluasi Model
y_proba = model.predict(X_test, verbose=0).ravel()
y_pred = (y_proba >= 0.5).astype(int)
print("Akurasi :", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
    target_names=le.classes_))
```

```
Akurasi : 0.8
Classification Report:
              precision    recall  f1-score   support

   negative      0.75      1.00      0.86         3
   positive      1.00      0.50      0.67         2

   accuracy              0.80         5
  macro avg              0.88         5
 weighted avg              0.85         5
```

6. Test Your Predictions on New Data

```
# Prediksi pada Data Baru
samples = [
    "I absolutely loved it",
    "This was awful and I regret watching it",
    "Not bad, could be better"
]
X_new = vectorizer(np.array(samples)).numpy()
preds = (model.predict(X_new, verbose=0).ravel() >= 0.5).astype(int)
for s, p in zip(samples, preds):
    print(f"- {s} -> {le.inverse_transform([p])[0]}")
```

```
- I absolutely loved it -> negative
- This was awful and I regret watching it -> negative
- Not bad, could be better -> positive
```

Congratulations! We've built a simple text classification model. Keep in mind that the model's accuracy may not yet be optimal because we're using a small dataset; ideally, deep learning works more effectively with larger datasets.



CODELAB (10%)

Complete the code below according to the instructions provided!

Dataset: [CODELAB DATASET](#)

1. Import Required Libraries. (10 Points)

```
import ... as np
import ... as pd
import ... as plt
import ... as sns
import ... as tf

from ... import StandardScaler, LabelEncoder
from ... import train_test_split
from ... import layers, models
from ... import classification_report, confusion_matrix, accuracy_score
```

2. Load Dataset. (10 Points)

```
df = pd.read_csv('...')

# Tampilkan 10 data awal
df...

# Tampilkan informasi ringkas data
df...

# Tampilkan statistik deskriptif data
df...
```

3. Separate the feature variables and the target variable. (10 Points)

Terms and Conditions:

- Features = all variables other than 'Species'
- Target = 'Species'

```
features = ... # semua kolom kecuali 'Species'
target = ... # kolom target
```

4. Data Splitting and Data Standardization. (20 Points)

Use a test size of 30% and a random state of 42.

```
# Splitting Data
X_train, X_test, y_train, y_test = ...(..., ..., test_size=..., random_state=...)

# Standarisasi Data
scaler = ...
scaler.fit(...)

X_train_scaled = scaler...(...)
X_test_scaled = scaler...(...)
```



5. Training the RNN Model. (20 Points)

```
num_classes = ...

model = models.Sequential([
    layers.Input(shape=...),
    layers.SimpleRNN(..., activation="tanh"),
    layers.Dense(..., activation="relu"),
    layers.Dense(..., activation="softmax")
])

model.compile(...="adam",
              ...="sparse_categorical_crossentropy",
              ...=["accuracy"])

model.summary()

history = model.fit(
    ..., ...,
    validation_split=0.2,
    epochs=200,
    batch_size=16,
    verbose=0
)
```

6. Model Evaluation. (30 Points)

Terms and Conditions:

- Create a confusion matrix
- Create a classification report
- Display the model's accuracy

```
print("Confusion Matrix:\n", ...(..., ...))
print("\nClassification Report:\n", ...(..., ...))
print("Accuracy:", ...(..., ...))
```



DETAILS OF CODELAB'S WORK

DETAIL	POINT
Import Library	10
Load Dataset	10
Separate the feature variables and the target variable	10
Data Splitting and Data Standardization	20
Training the RNN Model	20
Model Evaluation	30
TOTAL	100



LAB ASSIGNMENT (90%)

Dataset : [\[ASSIGNMENT DATASET\]](#)

1. Import the Library and Load the Dataset (5%)

- Import all Python libraries required for this task, such as pandas, numpy, scikit-learn, TensorFlow/Keras, and matplotlib. (30 Points)
- Display the top 5 and bottom 5 data points from the dataset. (35 Points)
- Display a statistical summary of the dataset, including the mean, median, standard deviation, minimum value, maximum value, and quartiles. (35 Points)

2. Data Preprocessing (10%)

Perform preprocessing on the tweet text, including:

- Remove URLs, mentions (@), and hashtags (#). (25 points)
- Text tokenization. (25 points)
- Stopword removal. (25 points)
- Lemmatization or stemming. (25 points)

3. Input and Target Class Initialization (10%)

- Split the data into features (input) and targets (labels). (50 points)
- Split the data into a training set (80%) and a validation set (20%). (50 points)

4. Build a Recurrent Neural Network (RNN) Model (10%)

- Design and build an appropriate model. (30 points)
- Specify the number of layers, neurons, and activation functions to be used. (35 points)
- Train the model using the training data. (35 points)

5. Model Evaluation (10%)

- Plot the accuracy and loss from the model training results. (25 points)
- Create a classification report along with label descriptions. (25 points)
- Test the model on new data (using test.csv). (25 points)
- Achieve a minimum accuracy of 85%. (25 points)

Note: Try to reach an accuracy level higher than 85%, and explain your method for improving accuracy to the assistant.



FINAL GRADE WEIGHTING

Activity	Description	Weight
Codelab		10%
Lab Assignment	Import the Library and Load the Dataset	5%
	Data Preprocessing	10%
	Input and Target Class Initialization	10%
	Build a Recurrent Neural Network (RNN) Model	10%
	Model Evaluation	10%
Explanation to Assistant		45%
Total		100%

